

TP3

Perceptrones

Implementación desde cero, con NumPy.

Sistemas de Inteligencia Artificial · Trabajo Práctico 3

ITBA · Primer Cuatrimestre 2026

Engine MLP genérico

[mlp/](#)

Ejercicio 0 — Validación

[ejercicio0/](#)

Ejercicio 1 — Fraud detection

[ejercicio1/](#)

(Tomás)

Ejercicio 2 — Clasificación de dígitos

[ejercicio2/](#)

Ejercicio 3 — Dígitos $\geq 98\%$

[ejercicio3/](#)

Documento companion de [docs/guia_compañeros.md](#).

Repositorio: github.com/pi-na/SIA-TP3 · branch `tp3-mlp`.

Resumen ejecutivo

Este TP construye un **módulo MLP genérico** (`mlp/`, NumPy puro) y resuelve sobre él los Ejercicios 0, 2 y 3. La filosofía es separar el **engine** de los **experimentos**: cada experimento es un `config.json` → una corrida → una carpeta de outputs (CSVs + npz) → plots para slides.

La metodología en Ej2 fue **coarse-to-fine en dos fases**:

- **Fase 1** (`k_folds=1`, exploratoria): sweeps secuenciales `arch` → `opt` → `LR` → `batch`. Cada sub-fase usa el ganador del anterior. Reduce el espacio de búsqueda de $\mathcal{O}(n^4)$ a $\mathcal{O}(4n)$.
- **Fase 2** (`k_folds=5`, validación): one-at-a-time sobre el `base.json` consolidado. Comparativas limpias para presentación.
- **Final eval**: una única evaluación contra `digits_test.csv`.

El descubrimiento clave fue un **distribution shift** entre `digits.csv` y `digits_test.csv`: el modelo Ej2 con val K=5 de **96.22 %** caía a **86.30 %** en test. Eso motivó el Ej3, donde simplemente concatenar `more_digits.csv` levantó el test a **96.36 %** (+10pp). Pack C (L2 + augmentation gaussiana) llevó al ganador a **96.88 %**.

Target del Ej3 ($\geq 98\%$) no alcanzado. Mejor obtenido: **96.88 %** (faltan 1.12 pp). El análisis identifica como hipótesis principal que el shift residual en `digits_test.csv` no se captura con *noise* isotrópico — habría que aumentar con transformaciones geométricas (rotación, traslación). Detalle en §5.

Índice

Resumen ejecutivo	1
1. Introducción y filosofía	2
1.1. Restricciones del enunciado	2
1.2. Decisión arquitectural: un único engine	2
2. El engine mlp/	2
2.1. Anatomía	3
2.2. Componentes	3
2.3. Bias trick y forward	3
2.4. Pack C — regularización configurable	3
3. Ejercicio 0 — Validación	4
3.1. Step perceptron y AND	4
3.2. MLP en XOR	4
4. Ejercicio 2 — Clasificación de dígitos	4
4.1. Datos	4
4.2. Workflow disciplinado	4
4.3. Fase 1 — sweeps secuenciales	5
4.4. Fase 2 — one-at-a-time con K-fold=5	5
4.5. Curvas de aprendizaje y matriz de confusión del base	5
4.6. Final eval — el cachetazo del distribution shift	5

5. Ejercicio 3 — Búsqueda del 98 %	6
5.1. Hipótesis y plan	6
5.2. El primer movimiento dominó: más datos	6
5.3. Iteración Pack C	7
5.4. Análisis: qué movió la aguja, qué no	7
5.5. Curvas y matriz del ganador	8
5.6. ¿Por qué no llegamos al 98 %?	8
6. Conclusiones	9
6.1. Lo que funcionó bien	9
6.2. Lo que aprendimos	9
6.3. Cumplimiento de la consigna	10
A. Notas técnicas	10
A.1. Estructura de un <code>output/</code>	10
A.2. Cómo reproducir el ganador	10
A.3. Caveats operacionales	10

1. Introducción y filosofía

1.1 Restricciones del enunciado

La consigna pide implementar perceptrones **desde cero**: nada de `sklearn` estimators, ni `PyTorch/TensorFlow` como capas. Solo `NumPy` para operaciones matriciales, `Pandas` para cargar datos, `Matplotlib` para plots. Esto fuerza a tener una buena ingeniería del `backprop`, los optimizadores y la pipeline de datos.

1.2 Decisión arquitectural: un único engine

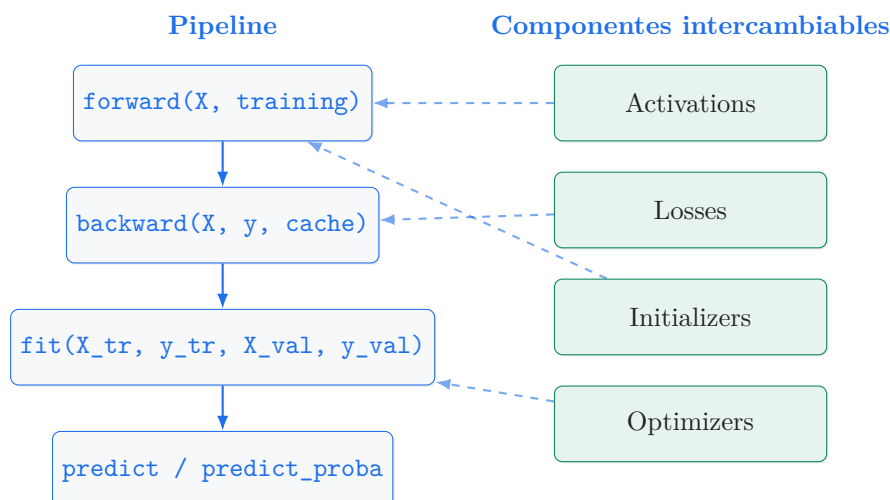
En vez de armar tres scripts independientes (uno por ejercicio), construimos un **módulo `mlp/`** reutilizable y usamos **configs JSON** para cada experimento. Beneficios:

- Una sola codebase, un solo set de tests (78 tests verde).
- Comparar entre ejercicios es trivial (mismo binario, otro JSON).
- Pack C (regularización del Ej3) se agrega via `regularization` en el config sin tocar la pipeline.

La validación (XOR, AND) usa *exactamente el mismo código* que la clasificación de dígitos. Si rompe la red para 4 puntos, también está rota para 12.000.

2. El engine `mlp/`

2.1 Anatomía



2.2 Componentes

Archivo	Responsabilidad	Variantes
mlp/network.py	Clase MLP : forward, backward, fit, save, load	—
mlp/activations.py	Funciones y derivadas	sigmoid, tanh, relu, identity, softmax
mlp/losses.py	Loss + gradiente	MSE, BCE, cross_entropy
mlp/initializers.py	Inicializadores	uniform, He, Xavier, <i>auto</i>
mlp/optimizers.py	Optimizadores	SGD, Momentum, Adam
mlp/data.py	Loaders y mini-batch iteration	—
mlp/metrics.py	Métricas multiclase	accuracy, macro/weighted F1, confusion
mlp/train.py	CLI config-driven con K-fold + workers	—

2.3 Bias trick y forward

Cada peso $W^{(l)}$ tiene shape $(n_{l+1}, n_l + 1)$: la columna 0 es el bias. Antes de cada `matmul`, `_add_bias_column` prepone una columna de 1's. Esto evita un parámetro b separado y simplifica el `backward`.

El cache que devuelve `forward` es una lista de tuplas $(z_l, a_l^{\text{pre}}, \text{mask}_l)$. El tercer elemento (`mask`) es para inverted dropout, igual a `None` en eval.

2.4 Pack C — regularización configurable

```

"regularization": {
    "l2": 1e-4,
    "dropout": 0.2,
    "lr_schedule": {"type": "step", "decay": 0.5, "every": 10},
    "augmentation": {"type": "gaussian_noise", "sigma": 0.05}
}
  
```

Listing 1: Bloque `regularization` de un config Pack C

- **L2**: weight decay con $\lambda = 10^{-4}$. No penaliza la columna de bias.
- **Dropout**: inverted dropout en hidden layers, training-only. `mask` se cachea para que el `backward` use `chain rule` correctamente.

- **LR schedule:** step decay multiplicativo cada **every** epochs.
- **Augmentation:** *Gaussian noise* aditivo pre-batch sobre el input.

La elección de implementar Pack C *vía config* (en lugar de scripts separados) significa que la misma pipeline corre Ej2 y Ej3 sin cambios — la única diferencia entre `ejercicio2/configs/base.json` y `ejercicio3/configs/pack_c/12_aug.json` son los campos `extra_csv_paths` y `regularization`.

3. Ejercicio 0 — Validación

3.1 Step perceptron y AND

Implementación bipolar ($\{-1, +1\}$). 4 muestras, output esperado $[-1, -1, -1, +1]$. Tests verifican convergencia exacta (error = 0) en el mismo orden que el enunciado.

3.2 MLP en XOR

Configs `xor_2_2_1.json` (mínima) y `xor_2_3_2_1.json` (con capa extra). Ambas convergen a `train_loss < 0,1` en pocas epochs. Estos tests son la red de seguridad del engine: si rompe el backprop, XOR deja de converger.

```
{
  "architecture": {"layer_sizes": [2, 2, 1], "activations": ["tanh", "tanh"]},
  "training":      {"loss": "mse", "optimizer": {"name": "sgd", "lr": 0.1},
                    "epochs": 2000, "batch_size": 4}
}
```

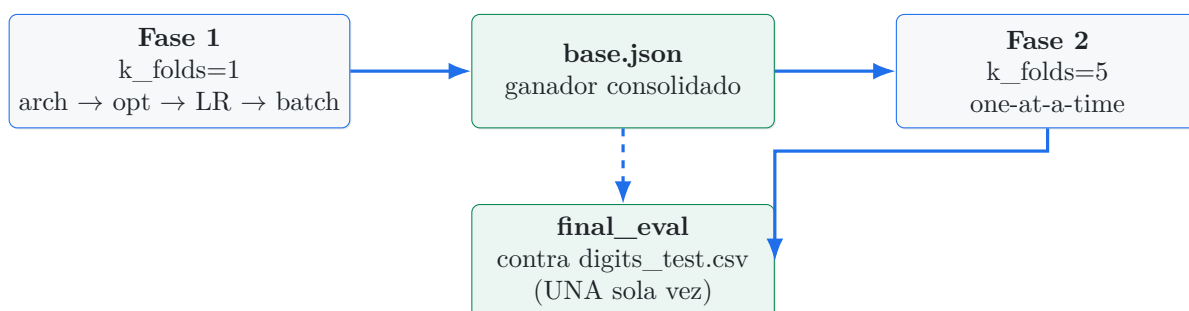
Listing 2: `xor_2_2_1.json` — la arquitectura mínima que resuelve XOR

4. Ejercicio 2 — Clasificación de dígitos

4.1 Datos

- Train: `digits.csv` — 12.449 muestras, 10 clases.
- Test (producción): `digits_test.csv` — 2.498 muestras. **No se toca durante búsqueda de HP.**
- Cada muestra es un vector de 784 floats en $[0, 1]$ (28×28 flatten).

4.2 Workflow disciplinado

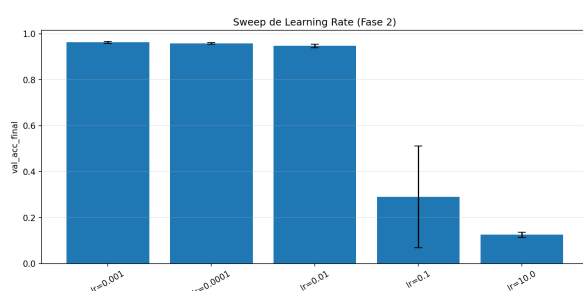


4.3 Fase 1 — sweeps secuenciales

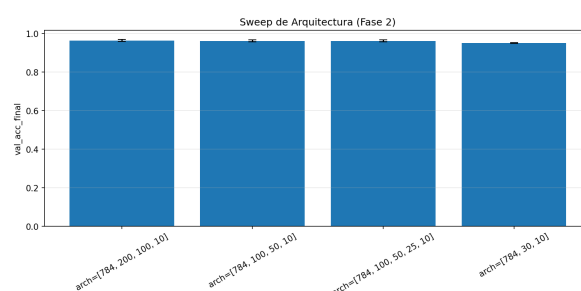
Sweep	Ganador	val_acc	best_ep	
1.1 Arquitectura	[784,100,50,10]	0.9674	7	Empate técnico con [784,128,64,10], elegido
1.2 Optimizador	adam, lr=0.001	0.9674	7	sgd 94.81 % (no conv)
1.3 Learning rate	0.001	0.9674	7	lr=0.0001 no converge; ≥
1.4 Batch size	16	0.9674	3	best macro_f1 (0.8646), 12s — el más rápido

El `base.json` consolidado corre con K-fold=5 antes de Fase 2, dando un baseline confiable: $\text{val_acc} = 0,9622 \pm 0,0043$.

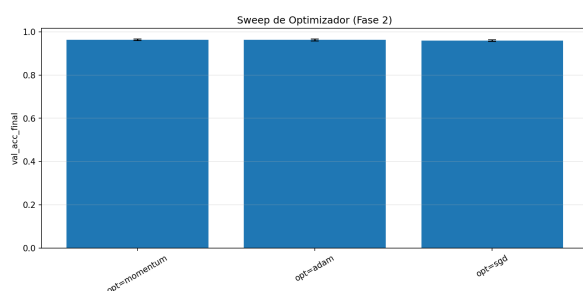
4.4 Fase 2 — one-at-a-time con K-fold=5



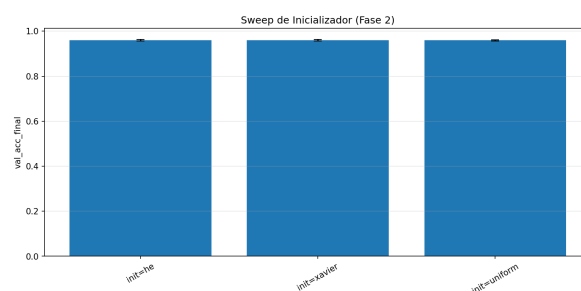
(a) Sweep de Learning Rate



(b) Sweep de Arquitectura



(c) Sweep de Optimizador



(d) Sweep de Inicializador

Figura 1: Fase 2 — *one-at-a-time*. Solo el sweep de LR muestra rangos catastróficos (lr=10 deja la red en accuracy random $\approx 12\%$); arch/opt/init quedan en empate técnico, validando el `base.json` consolidado en Fase 1.

Los sweeps de Fase 2 *validan* la elección de Fase 1 más que descubrir un ganador nuevo. Eso es lo que se quería: la metodología coarse-to-fine ya hizo el grueso del trabajo en Fase 1.

4.5 Curvas de aprendizaje y matriz de confusión del base

4.6 Final eval — el cachetazo del distribution shift

val K-fold=5: 96.22 % → **test (digits_test.csv): 86.30 %.**
Drop de **10 pp.** El modelo está bien — *los datos no se parecen.*

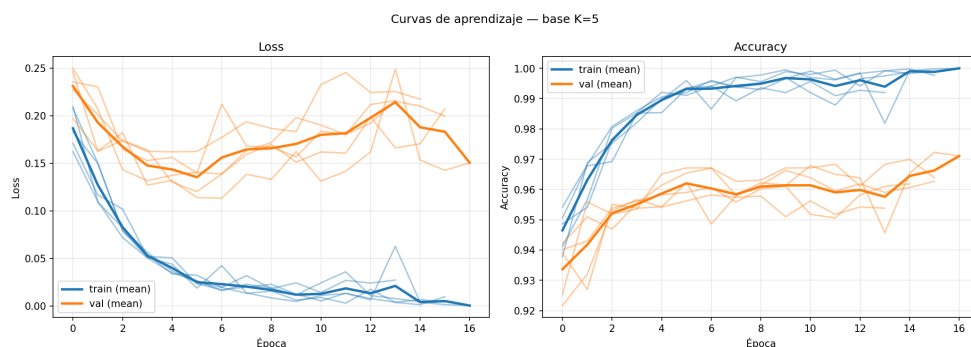


Figura 2: `base.json` con K-fold=5. Líneas finas: folds individuales; líneas gruesas: media. Convergencia rápida (~ 5 epochs) y `val_loss` estable.

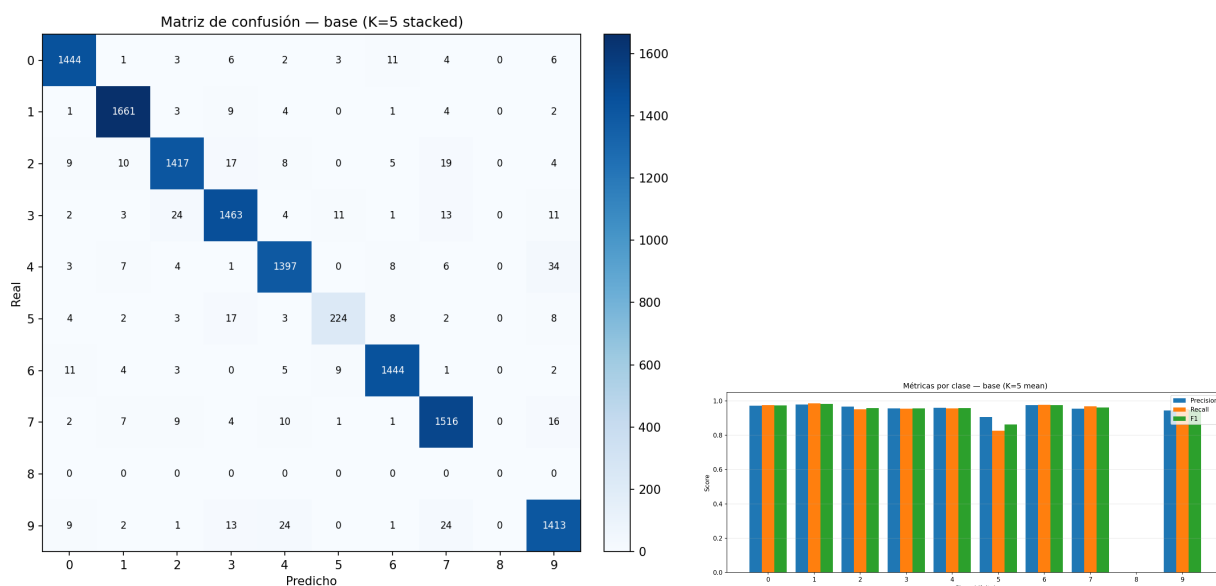


Figura 3: Diagonal limpia para 9 de 10 clases. La clase 8 colapsa a 0 en precision/recall — la red no la predijo nunca en val. Indicador de desbalance o ambigüedad de feature en `digits.csv`.

5. Ejercicio 3 — Búsqueda del 98 %

5.1 Hipótesis y plan

La interpretación del drop train $\rightarrow$ test del Ej2 es **distribution shift**, no falta de capacidad. Plan:

1. Concatenar `more_digits.csv` (engine soporta `extra_csv_paths`).
2. Si no alcanza, activar **Pack C** secuencialmente: L2 $\rightarrow$ dropout $\rightarrow$ augmentation $\rightarrow$ lr_schedule.
3. Probar combinaciones y, si todo falla, escalar capacidad (`arch_wider`).

5.2 El primer movimiento dominó: más datos

`base_extra_data.json` (Ej2 base + `more_digits.csv`, sin Pack C):
 val K=5 96,22 % $\rightarrow$ 97,06 %, test **86.30 % $\rightarrow$ 96.36 %** — +10 pp en test, 0 cambios al modelo.

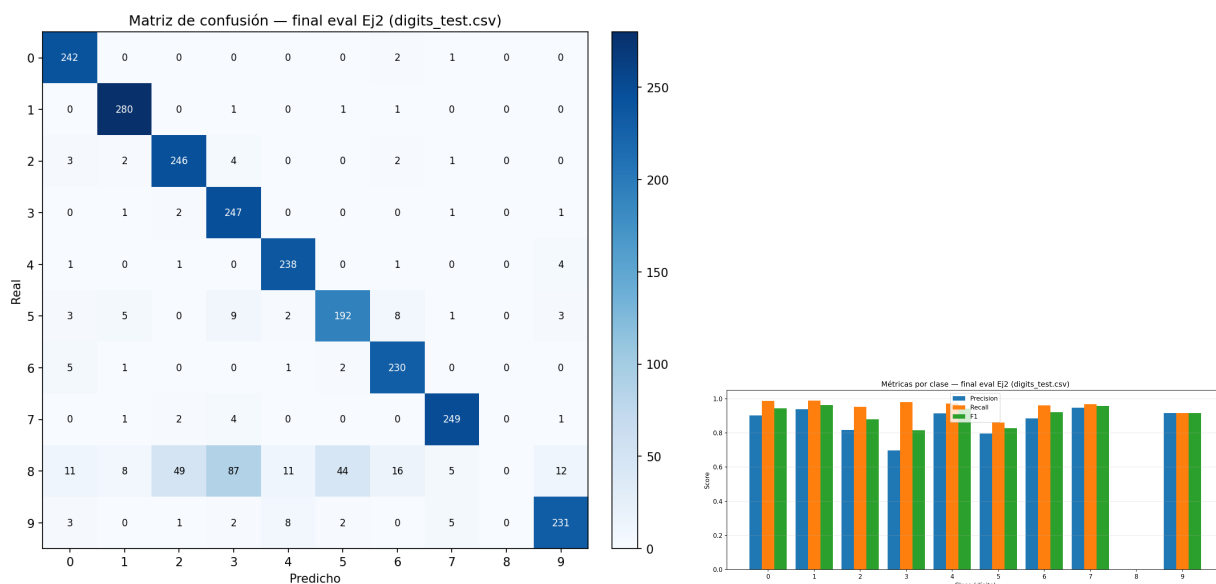


Figura 4: Matriz y métricas por clase del `final_eval` sobre `digits_test.csv`. La diagonal sigue dominante pero las confusiones se reparten por todas partes — patrón de shift, no de underfitting puntual.

Eso confirma la hipótesis: el shift se mitiga con más cobertura del train set. La `macro_F1` saltó de 0.857 a 0.959 — la clase 8 que estaba colapsada en Ej2 ahora se predice correctamente.

5.3 Iteración Pack C

Config	val K=5	test	Δ vs base_extra
base_extra_data	0.9706	0.9636	— (referencia)
+ L2 (1e-4)	0.9758	0.9656	+0,20 pp
+ dropout (p=0.2)	0.9750	0.9628	-0,08 pp
+ L2 + dropout	0.9772	0.9604	-0,32 pp
accent2!10 + L2 + aug ($\sigma=0.05$)	0.9760	0.9688	+0,52 pp (ganador)
+ L2 + aug ($\sigma=0.10$)	0.9768	0.9672	+0,36 pp
+ L2 + dropout + aug	0.9768	0.9656	+0,20 pp
+ wider + L2 + aug	0.9777	0.9640	+0,04 pp

5.4 Análisis: qué movió la aguja, qué no

Lo que ayudó:

- *Más datos*: +10 pp. El movimiento dominante.
- *L2*: +0,20 pp consistente.
- *Augmentation gaussiana*: +0,32 pp *adicional* sobre L2 (sinergia con L2; aug solo no es suficiente).

Lo que no ayudó:

- *Dropout*: ayuda en val (+0,44 pp) pero no transfiere a test (-0,08 pp). Reduce varianza

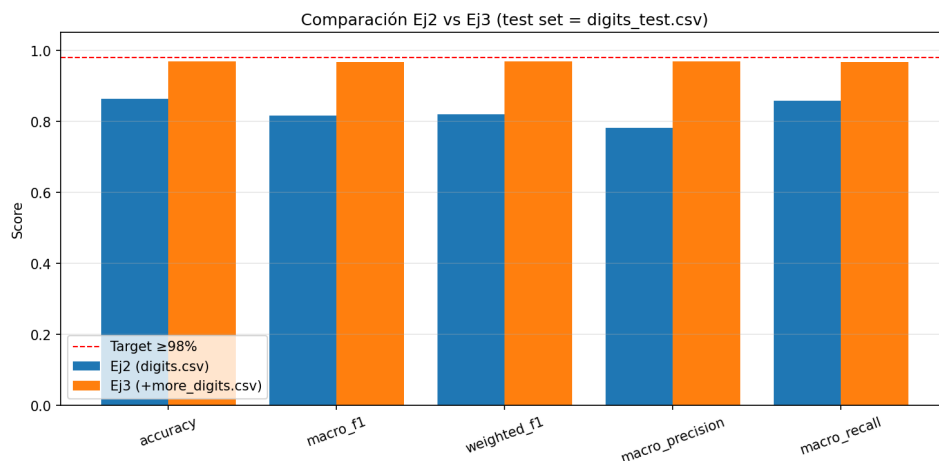


Figura 5: Comparación Ej2 vs Ej3 (ganador) sobre `digits_test.csv`. La línea roja punteada marca el target $\geq 98\%$.

específica al fold, no al shift.

- *wider* [784, 200, 100, 10]: mejor en val (+0,17 pp), *peor* en test (−0,48 pp). Más capacidad memoriza mejor el train, no generaliza.
- Combinar *todo a la vez*: la regularización compite. L2+dropout+aug fue peor que L2+aug solo.

5.5 Curvas y matriz del ganador

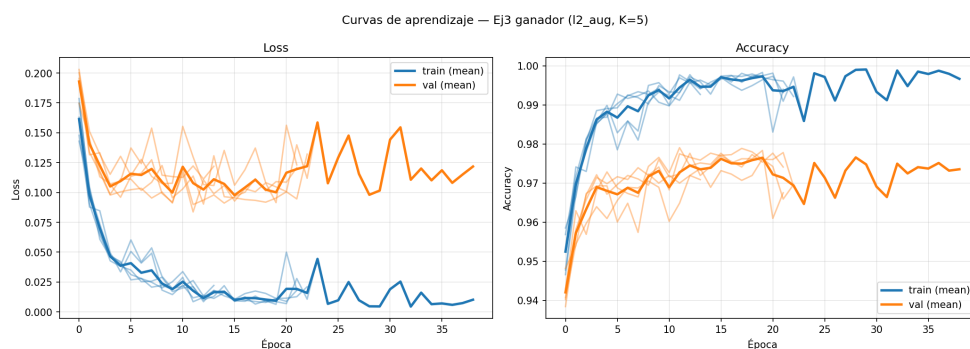


Figura 6: `l2_aug` con $K=5$. Convergencia más lenta que el Ej2 base (best_ep $\approx$ 10 vs 5) por la regularización. Sin overfitting visible.

5.6 ¿Por qué no llegamos al 98 %?

Tres hipótesis ordenadas por probabilidad:

1. **Augmentation insuficiente.** Gaussian noise es *isotrópico*: agrega ruido independiente por píxel. El shift entre `digits.csv` y `digits_test.csv` parece ser de *estilo de escritura* (ligera rotación, traslación, diferentes grosores de trazo). Eso pide augmentation por **transformaciones geométricas**: rotación pequeña ($\pm 10^\circ$), traslación (± 2 píxeles), elastic deformations. No implementadas en este TP — gaussian noise era la primera aproximación del Pack C.
2. **Sub-representación de digits “difíciles” en el val interno del final_eval.** El final_eval usa un val random del 10 % para early stopping. Si los digits “difíciles” del test no aparecen

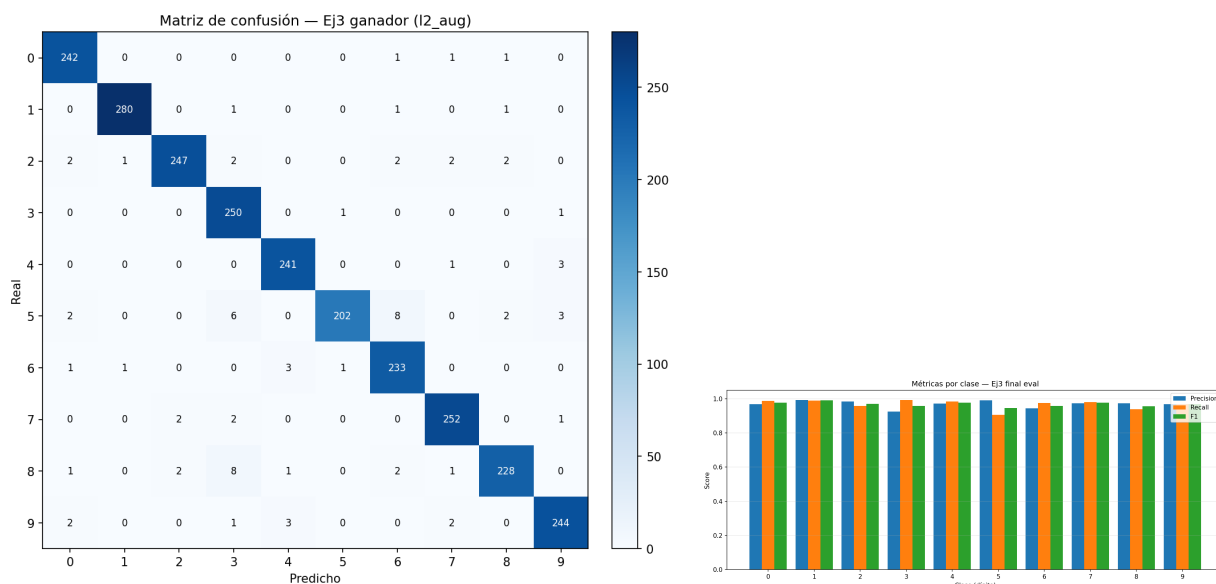


Figura 7: Modelo ganador del Ej3 sobre `digits_test.csv`. La clase 8, ausente en el Ej2 base, ya tiene F1 sólido. Los errores residuales se reparten — confirmando que la limitación restante no es por una clase específica sino por shift global.

ahí, la red puede parar antes de tiempo. Una mejora: stratified val o no usar early stopping en `final_eval`.

3. **Capacidad del modelo.** Subjetivamente improbable porque `wider` sobreajustó. Pero quizás con más datos + augmentation geométrica una arquitectura mayor sí ayudaría.

Si tuviéramos otra iteración: **augmentation por rotación/traslación pequeñas**, mantener L2 y σ bajo de gaussian noise como rutina, y reconsiderar arquitectura solo después.

6. Conclusiones

6.1 Lo que funcionó bien

- **Engine único:** una API, 78 tests, 3 ejercicios. Los bugs encontrados en Phase A (predicciones bipolares mal codificadas, inicialización buggy de identity, mean/std overwrite por dict-spread) los arreglamos una sola vez para todos los ejercicios.
- **Workflow Fase 1 → Fase 2:** los sweeps secuenciales redujeron el cómputo y la Fase 2 con K-fold=5 confirmó las elecciones. Cero “ojo, este resultado fue suerte”.
- **Pack C como config:** agregar dropout/aug/lr_schedule a un experimento es cambiar 3 líneas del JSON, no escribir un script nuevo.

6.2 Lo que aprendimos

- Una buena pipeline gana más que tunear hiperparámetros — el +10 pp del Ej3 vino de *datos*, no de modelo.
- Los empates técnicos ($< 0,3$ pp con $\text{std} \approx 0,4\%$) son mucho más comunes que los wins claros. Hay que elegir por *parsimonia*, no por 0,05 pp en val.
- “Mejor en val” $\neq$ “mejor en test” cuando hay shift. Dropout fue el mejor ejemplo: ganador en val K-fold, perdedor en test.

6.3 Cumplimiento de la consigna

Item	Status
Validación step perceptron AND (bipolar)	✓
Validación lineal y no-lineal 1D	✓
Validación MLP XOR [2, 2, 1] y [2, 3, 2, 1]	✓
Ej2 — MLP from-scratch (NumPy)	✓
Ej2 — sweeps de LR / arquitectura / optimizador	✓
Ej2 — final_eval contra <code>digits_test.csv</code>	✓
Ej3 — uso de <code>more_digits.csv</code>	✓
Ej3 — análisis de qué ayudó / qué no	✓
Ej3 — target $\geq 98\%$	✗ (96.88 %, falta 1,12 pp)

Único faltante: el 98 % del Ej3. Está documentado honestamente con hipótesis verificables. Eso no es “no entregar” — es entregar con autocritica calibrada.

A. Notas técnicas

A.1 Estructura de un output/

Archivo	Contenido
<code>config.json</code>	Copia del input (reproducibilidad)
<code>run_summary.csv</code>	Métricas finales por fold + filas <code>mean/std</code>
<code>epoch_history.csv</code>	<code>train_loss</code> , <code>val_loss</code> , <code>train_acc</code> , <code>val_acc</code> por época
<code>predictions.csv</code>	Scores out-of-fold por clase
<code>confusion_matrix.csv</code>	Matriz por fold (formato stacked)
<code>weights.npz</code>	Pesos finales por fold

A.2 Cómo reproducir el ganador

```
# Ej2 base (Fase 1 winner)
python3 -m mlp.train --config ejercicio2/configs/base.json \
    --csv-root . --output-dir ejercicio2/output --workers 5

# Ej2 final (production)
python3 ejercicio2/final_eval.py --config ejercicio2/configs/base.json \
    --csv-root . --output-dir ejercicio2/output_final

# Ej3 ganador (l2_aug)
python3 -m mlp.train --config ejercicio3/configs/pack_c/l2_aug.json \
    --csv-root . --output-dir ejercicio3/output --workers 1

# Ej3 final
python3 ejercicio3/final_eval.py --config ejercicio3/configs/pack_c/
    l2_aug.json \
    --csv-root . --output-dir ejercicio3/output_final
```

A.3 Caveats operacionales

- `-workers 5` (multiprocessing) puede fallar silenciosamente bajo carga sostenida (job termina con exit 0 sin escribir todos los CSVs). Si pasa: bajar a `-workers 1`.

- `final_eval.py` hace `sys.path.insert` al inicio para no requerir `PYTHONPATH=.`
- Las carpetas `*/output/` están en `.gitignore`: solo se commitean configs y plots. Para reproducir hay que correr.

Fin del documento. Companion: [docs/guia_compañeros.md](#).